

DeepLabV3

[DeepLabv3+의 주요 특징과 각 특징의 장점]

먼저 DeepLabv3+의 주요 특징 먼저 나열하겠습니다.

- ASPP(Atrous Spatial Pyramid Pooling)
- encoder-decoder structure
- depthwise separable convolution

각 특징의 장점을 간략히 설명하면 아래와 같습니다.

1) ASPP(Atrous Spatial Pyramid Pooling)

기본적으로 ASPP를 사용하기 때문에

하나의 이미지를 다양한 크기의 조각에서 바라볼 수 있게 되므로,

비교적 풍부한 문맥적인 정보를 추출할 수 있습니다.

DeepLabv3에서 쓰던 ASPP에 대한 설명은 여기서는 생략하도록 하겠습니다.

대신 Atrous convolution에 대한 내용은 아래에서 다시 다루도록 할게요.

(DeepLabv3 논문: <https://arxiv.org/pdf/1706.05587.pdf>)

(아래는 DeepLabv3에서의 ASPP 구조 사진)

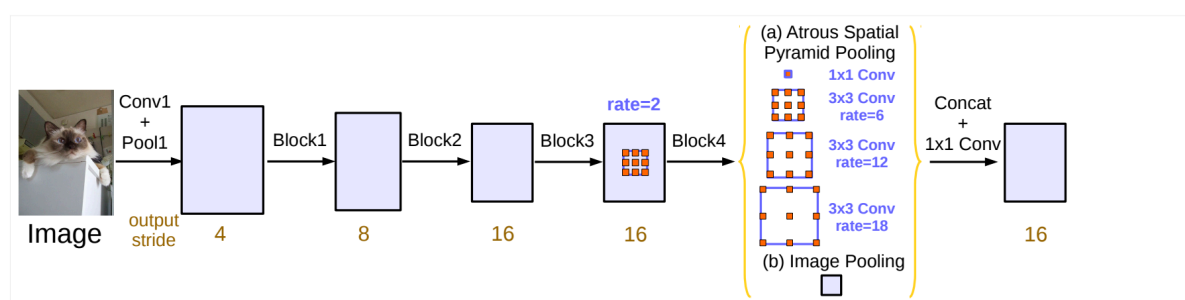


Figure 5. Parallel modules with atrous convolution (ASPP), augmented with image-level features.

출처: <https://arxiv.org/pdf/1706.05587.pdf>

2) encoder-decoder structure

encoder-decoder 구조를 사용하기 때문에

위에서 추출한 정보를 가지고 segmentation을 진행할 시,

물체의 경계가 모호하지 않도록 공간적인 정보를 최대한 유지할 수 있게 됩니다.

기본적으로 semantic information(=encoder의 output)은

encoding시에 pooling과 striding을 주는 convolution으로 인해

물체 경계와 관련된 정교한 정보들을 잃게 됩니다.

그렇다면 누군가는

'feature map(=semantic information)을 더 정교하게 만들면 되지 않나?'

라는 질문을 할 수 있을 겁니다.

하지만 논문에서는 (논문이 나온 시점에서) 성능이 좋은 neural network의 구조와

한정된 GPU 메모리를 고려했을 때,

feature map(encoder의 output)의 크기는 input 사진의 크기보다

8배이상 더 작아야 된다고 합니다.

아래 사진에서 왼쪽에서 3번째 사진은

위에서 언급한 ASPP와 encoder-decoder structure이 동시에 적용된 사진입니다.

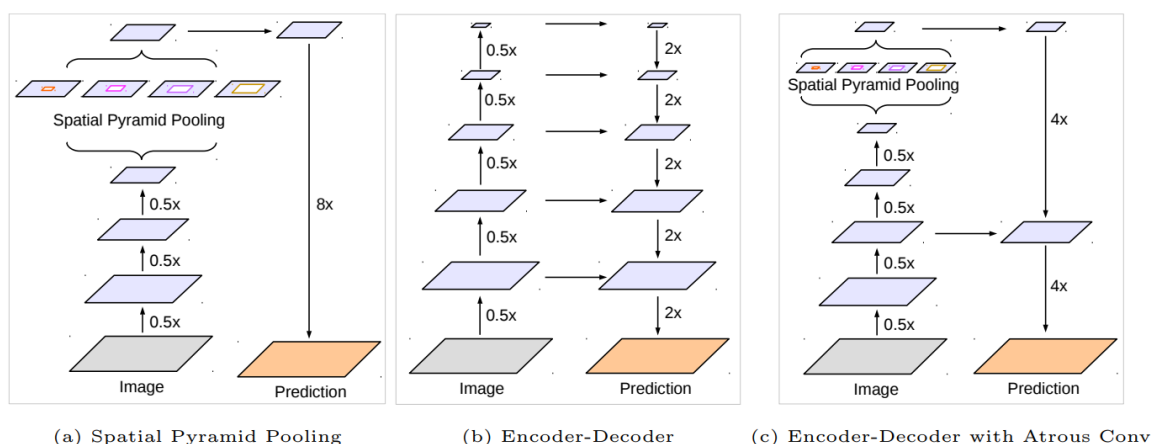


Fig. 1. We improve DeepLabv3, which employs the spatial pyramid pooling module (a), with the encoder-decoder structure (b). The proposed model, DeepLabv3+, contains rich semantic information from the encoder module, while the detailed object boundaries are recovered by the simple yet effective decoder module. The encoder module allows us to extract features at an arbitrary resolution by applying atrous convolution.

출처: <https://arxiv.org/pdf/1802.02611.pdf>

3) depthwise separable convolution

depthwise separable convolution을 활용하기 때문에

정확도와 속도의 trade-off를 해결할 수 있습니다.

논문의 저자는 (논문이 나온 시점에서) 최근 depthwise separable convolution이 성공을 이뤘기 때문에,

semantic segmentation에서 사용하기에 적합하고

속도와 정확도 두가지 측면에서 모두 이점을 가지도록

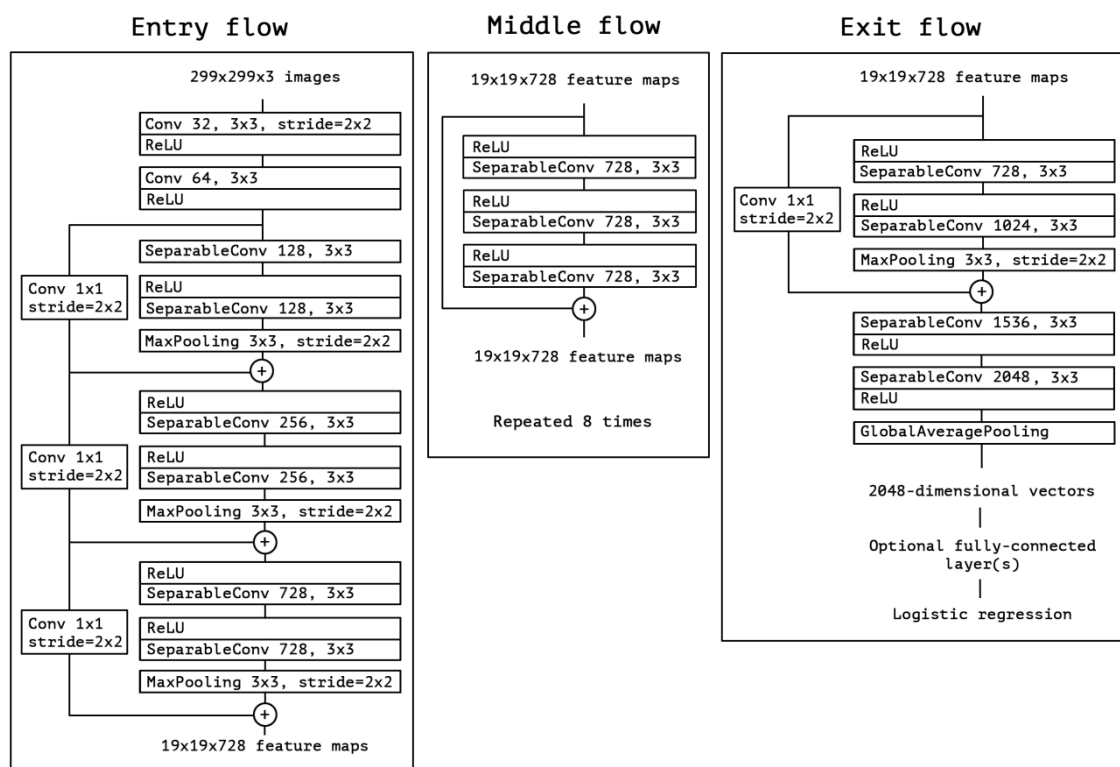
기존 Xception model에 depthwise separable convolution을 적용했다고 합니다.

Xception model에 대한 내용은 여기서는 설명하지 않겠습니다.

(Xception 논문: <https://arxiv.org/pdf/1610.02357.pdf>)

(아래 사진은 Xception architecture)

Figure 5. The Xception architecture: the data first goes through the entry flow, then through the middle flow which is repeated eight times, and finally through the exit flow. Note that all Convolution and SeparableConvolution layers are followed by batch normalization [7] (not included in the diagram). All SeparableConvolution layers use a depth multiplier of 1 (no depth expansion).



출처: <https://arxiv.org/pdf/1610.02357.pdf>

그리고 수정된 Xception 모델은 ASPP모듈에 적용시켰다고 합니다.

depthwise separable convolution에 대한 더 자세한 설명은

아래에 작성해두었습니다.

[Atrous convolution / Depthwise seperable convolution / Deepabv3 / Decoder / modified Xception]

부제목이 너무 길죠? ㅎㅎ

다른게 아니라 이 논문의 chapter 3에서는 위와 관련된 짹막한 설명을 해주기 때문에,

여기에서도 각 개념에 대해 짧게 정리해보는 시간을 가지려 합니다.

위의 **[DeepLabv3+의 주요 특징과 각 특징의 장점]**에서는

DeepLabv3+의 가장 대표되는 특징 3개와 이들의 장점에 대해 소개했다면, 이번에는 DeepLabv3+에 쓰이는 개념들에 대해 짧막하게 소개하는 과정이라고 생각하면 될 것 같습니다.

1. Atrous convolution

Atrous convolution은 DeepLab논문에서 처음 등장한 용어입니다.

아래 사진의 위쪽 그림은 1차원에서의 일반적인 convolution이고,

아래 그림은 1차원에서의 atrous convolution 입니다.

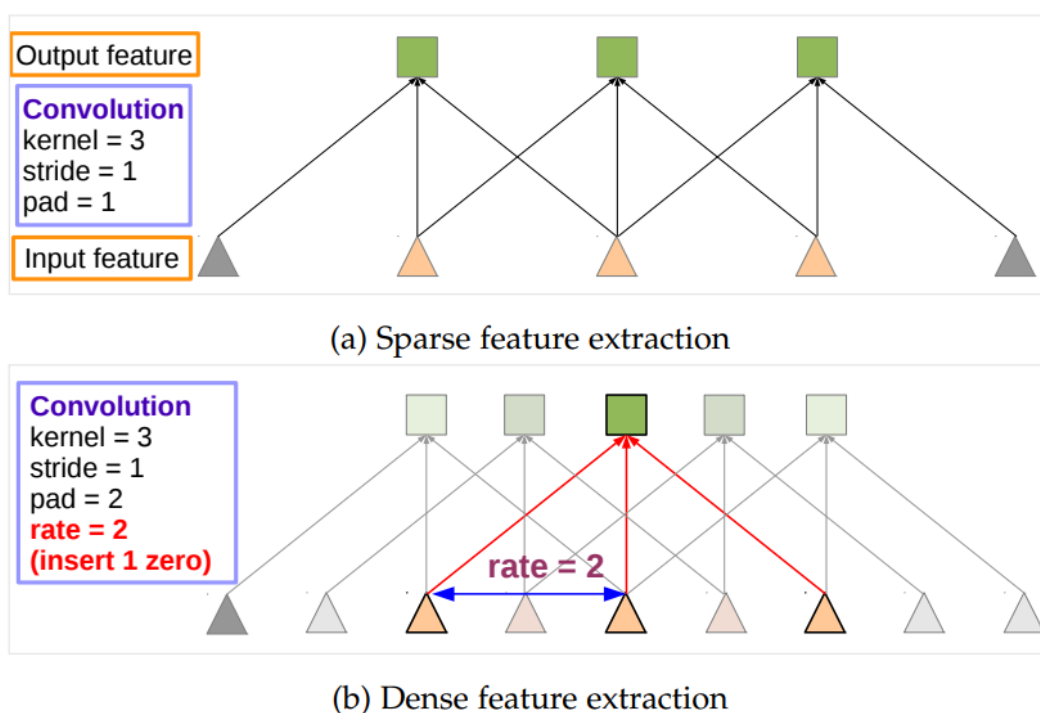


Fig. 2: Illustration of atrous convolution in 1-D. (a) Sparse feature extraction with standard convolution on a low resolution input feature map. (b) Dense feature extraction with atrous convolution with rate $r = 2$, applied on a high resolution input feature map.

출처: <https://arxiv.org/pdf/1606.00915.pdf>

이로 인해 receptive field의 영역이 커지는 효과를 줄 수 있습니다.

논문에서는 receptive field를 'field-of-view'라고 표현했네요.

Astrous convolution을 수식으로 표현하면 아래와 같습니다.

$$y[i] = \sum_k x[i + r \cdot k] w[k]$$

Handwritten annotations for the equation above:

- $y[i]$: output feature map
- x : input feature map
- k : kernel
- i : location
- r : atrous rate
- $w[k]$: convolution filter

여기서 $r=1$ 일때, 일반적인 convolution을 뜻하게 됩니다.

따라서 논문에서는 astrous convolution을 기존 convolution을 generalize한 버전이라고 표현하고 있습니다.

astrous rate값에 따라 receptive field의 크기가 달라지는것을 이용해,
(값이 클수록 receptive field 크기 또한 커지게 되겠죠.)

ASPP에서는 rate를 다양하게 하여 multi-scale 정보를 읽고 있습니다.

2. Depthwise separable convolution

Depthwise separable convolution은 일반적인 convolution을
두가지 과정을 통해 진행하는 것 입니다.

먼저 depthwise convolution을 진행하고,

pointwise convolution(1×1 convolution)을 진행하게 됩니다.

이것의 장점은 연산 복잡도를 크게 감소시켜준다는 것입니다.

아래 사진은 Depthwise separable convolution을 나타낸 것입니다.

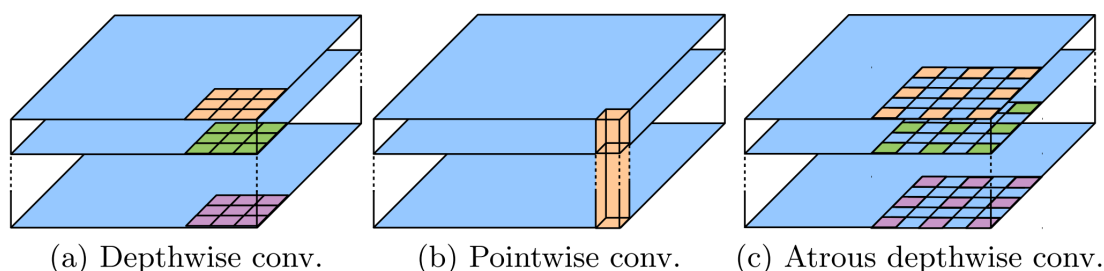


Fig. 3. 3×3 Depthwise separable convolution decomposes a standard convolution into (a) a depthwise convolution (applying a single filter for each input channel) and (b) a pointwise convolution (combining the outputs from depthwise convolution across channels). In this work, we explore *atrous separable convolution* where atrous convolution is adopted in the depthwise convolution, as shown in (c) with $rate = 2$.

출처: <https://arxiv.org/pdf/1802.02611.pdf>

그림에서도 알 수 있듯이,

depthwise convolution은 각 채널마다 특정 커널이 존재하고,

이 커널을 통해 해당 channel에 대해서만 convolution을 진행합니다.

그 후 pointwise convolution에서는

depthwise convolution의 결과물에 대해서 1×1 convolution을 진행하게 됩니다.

이 논문에서는 Astrous convolution에다 Depthwise separable convolution을 적용시켰고,

이를 astrous separable convolution이라고 부르고 있습니다.

astrous separable convolution을 적용 시에

모델의 성능은 비슷하게 유지되지만

연산 복잡도는 크게 감소시켰다고 합니다.

3. DeepLabv3 as encoder

위에서 언급했듯이 DeepLabv3+에서는 encoder-decoder구조를 사용하고 있습니다.

이 때 encoder에는 기존 DeepLabv3모델의 구조를 사용합니다.

참고로 DeepLabv3에서도 ASPP를 쓴다는 것을 염두하시길 바랍니다.

4. Proposed decoder

DeepLabv3+의 이전 버전인 DeepLabv3는 output stride가 16입니다.

(모델의 최종 값의 크기가 input에 들어가는 이미지 크기보다 16배 작다는 의미입니다.)

DeepLabv3는 이를 bilinearly upsample을 해서

input 이미지와 해상도를 같게 해줬는데

이 과정에서 세밀한 경계값들의 정보가 손실된다는 문제점이 있었습니다.

그래서 전에 반복해서 언급했듯이 DeepLabv3+에서는 decoder를 도입한거구요.

아래 그림은 DeepLabv3+ 구조를 나타냅니다.

파란색 큰 박스가 encoder, 빨간색 큰 박스가 decoder를 의미합니다.

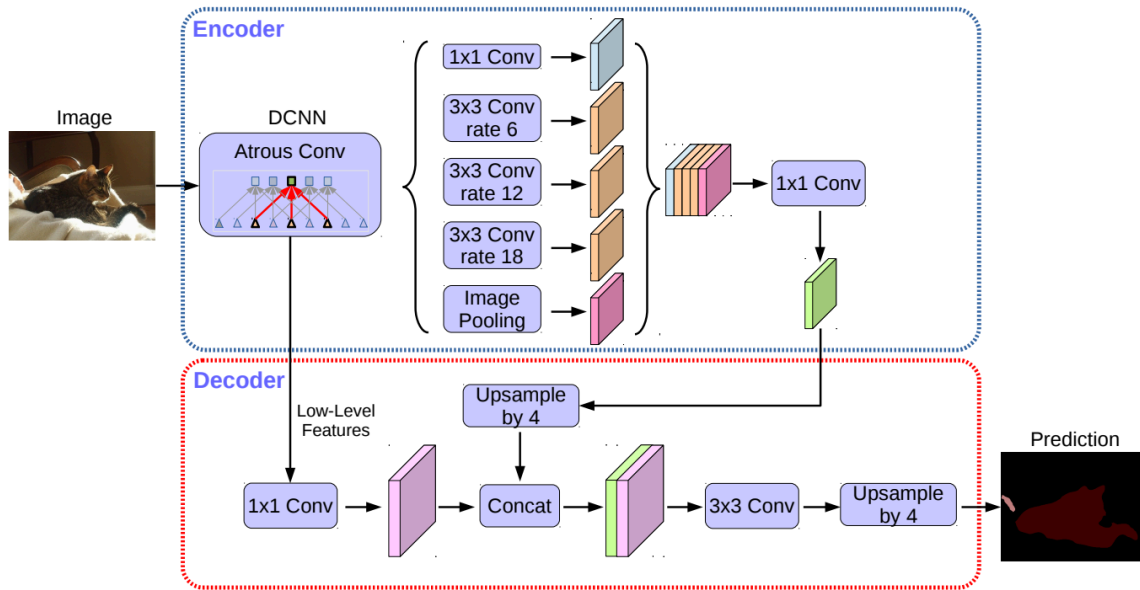


Fig. 2. Our proposed DeepLabv3+ extends DeepLabv3 by employing a encoder-decoder structure. The encoder module encodes multi-scale contextual information by applying atrous convolution at multiple scales, while the simple yet effective decoder module refines the segmentation results along object boundaries.

출처: <https://arxiv.org/pdf/1802.02611.pdf>

5. Modified Aligned Xception

DeepLabv3+에서는 기존 Aligned Xception모델을 semantic segmentation에 적합하도록 수정하여 사용했습니다.

바뀐 점은 아래와 같습니다.

(1)

기존 Xception의 entry flow network structure은 건드리지 않되, middle flow 부분을 더 깊게 만들었습니다.

(2)

모든 max pooling 연산을 stride를 가진 depthwise separable convolution으로 대체했습니다.

아마 정보의 손실을 막고자 이렇게 변경하지 않았나 싶습니다.

(3)

MobileNet구조와 비슷하게 3x3 depthwise separable convolution이 일어날 때마다, batch normalization과 ReLU를 수행해줬습니다.

위 그림의 DCNN 과정은

수정된 버전의 Aligned Xception을 통해 이뤄집니다.

아래 그림은 수정된 Xception 구조입니다.

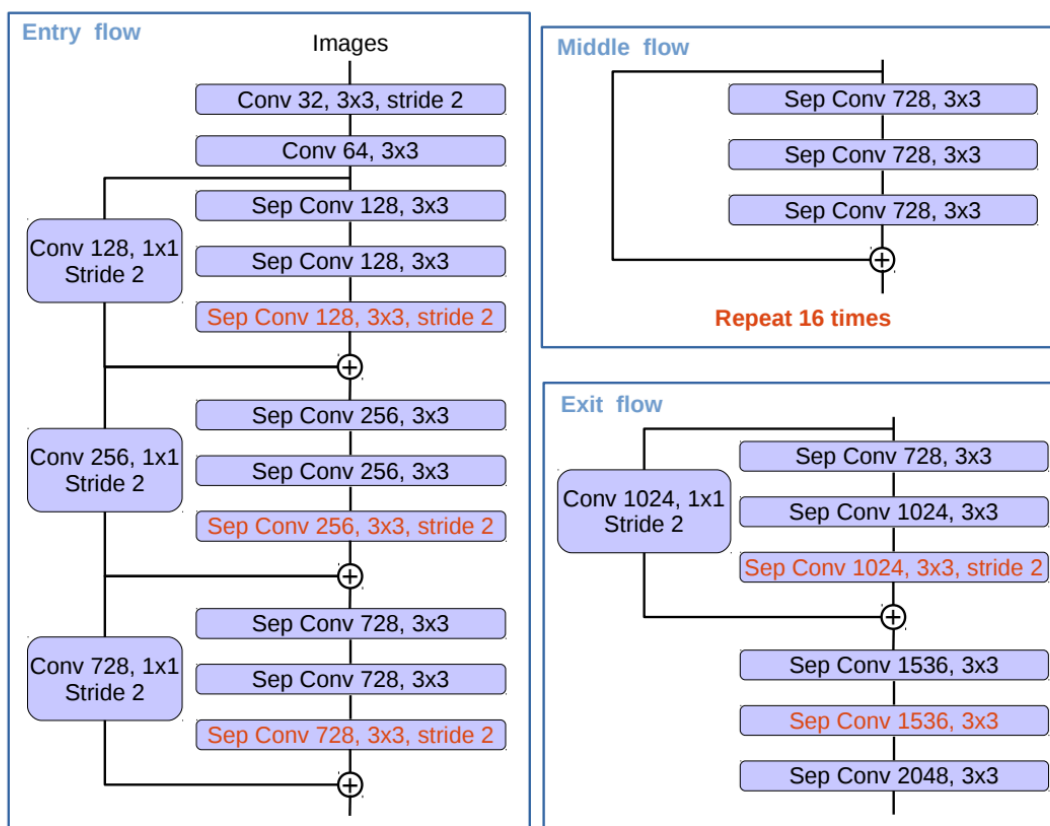


Fig. 4. We modify the Xception as follows: (1) more layers (same as MSRA's modification except the changes in Entry flow), (2) all the max pooling operations are replaced by depthwise separable convolutions with striding, and (3) extra batch normalization and ReLU are added after each 3×3 depthwise convolution, similar to MobileNet.

출처: <https://arxiv.org/pdf/1802.02611.pdf>

[여러 실험들]

4장 Experimental Evaluation에 등장한 실험에 대해 소개하겠습니다.

1. Decoder Design Choices

논문에서는 decoder가 어떤 구조일 때 좋은 성능을 내는지 소개하고 있습니다.

논문에서 실험한 바는 아래와 같습니다.

(1) encoder에서부터 온 low-level feature map의 channel을 몇으로 줄일 때 성능이 가장 좋은지

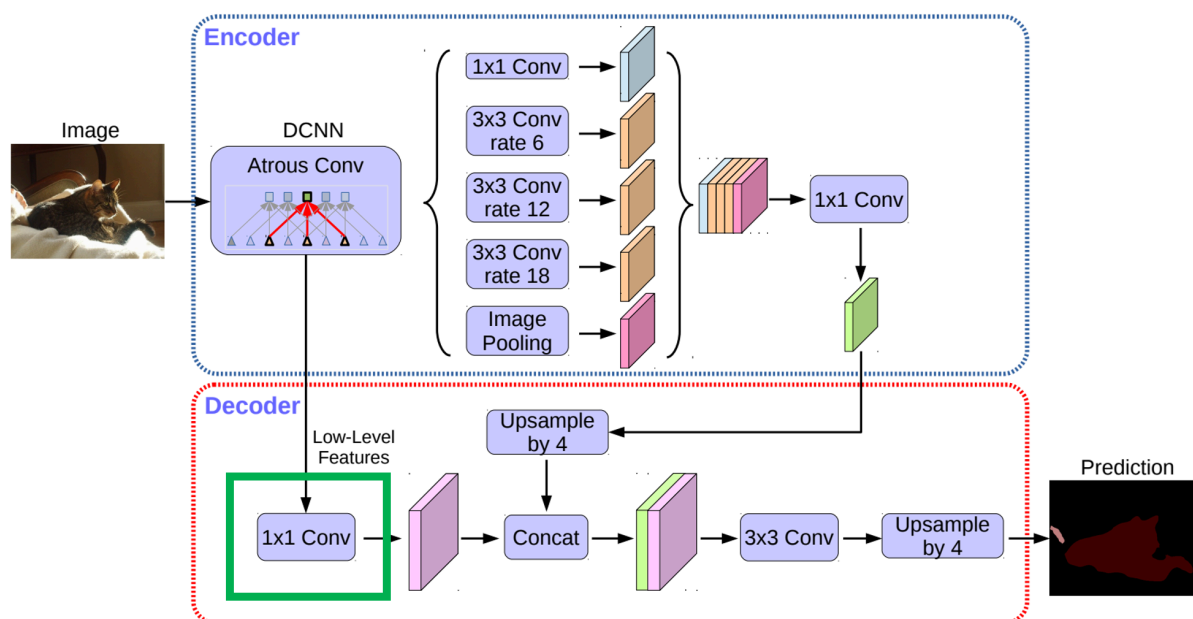
(2) 3×3 convolution 부분의 구조를 어떻게 했을 때 성능이 가장 좋은지

(1) encoder에서부터 온 low-level feature map의 channel을 몇으로 줄일 때 성능이 가장 좋은지

참고로 encoder에서부터 온 low-level feature map을

1×1 convolution을 통해 채널 수를 줄이는 과정은

아래 그림에서의 초록색 박스 부분입니다.



해당 실험의 결과는 아래 테이블과 같습니다.

Channels	8	16	32	48	64
mIOU	77.61%	77.92%	78.16%	78.21%	77.94%

Table 1. PASCAL VOC 2012 *val* set. Effect of decoder 1×1 convolution used to reduce the channels of low-level feature map from the encoder module. We fix the other components in the decoder structure as using $[3 \times 3, 256]$ and Conv2.

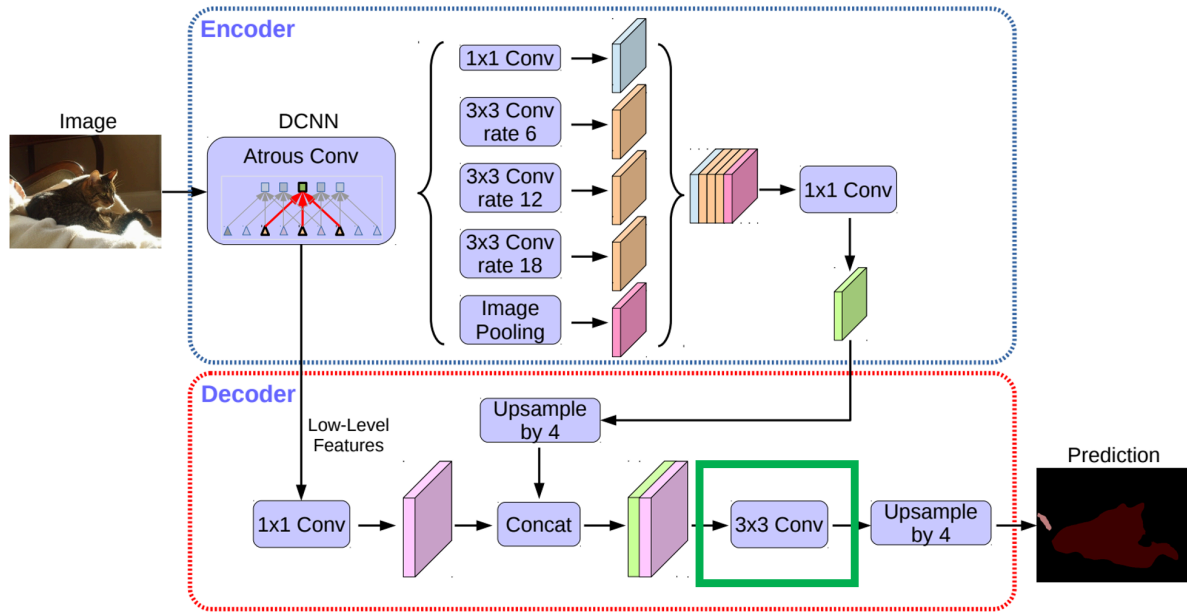
출처: <https://arxiv.org/pdf/1802.02611.pdf>

위의 사진에서 보다시피 채널을 48로 줄일 때 제일 좋은 성능을 보여주고 있습니다.

(2) 3×3 convolution 부분의 구조를 어떻게 했을 때 성능이 가장 좋은지

참고로 3×3 convolution 부분은

아래 그림에서의 초록색 박스 부분입니다.



해당 실험의 결과는 아래 테이블과 같습니다.

Features	3 × 3 Conv	mIOU
Conv2 Conv3	Structure	
✓	$[3 \times 3, 256]$	78.21%
✓	$[3 \times 3, 256] \times 2$	78.85%
✓	$[3 \times 3, 256] \times 3$	78.02%
✓	$[3 \times 3, 128]$	77.25%
✓	$[1 \times 1, 256]$	78.07%
✓	✓ $[3 \times 3, 256]$	78.61%

Table 2. Effect of decoder structure when fixing $[1 \times 1, 48]$ to reduce the encoder feature channels. We found that it is most effective to use the Conv2 (before striding) feature map and two extra $[3 \times 3, 256]$ operations. Performance on VOC 2012 *val* set.

출처: <https://arxiv.org/pdf/1802.02611.pdf>

위에서 보듯이 encoder의 Conv2 feature map과 encoder의 결과물을 concat한 것에 대해

$[3 \times 3, 256]$ convolution을 2번 했을 때가 제일 성능이 좋았습니다.

또한 encoder의 low-level feature를 Conv2이외에도 Conv3을 같이 썼을 때

Conv2만 썼을 때의 성능보다 안좋았기 때문에,

Conv2의 정보만 사용하기로 하였습니다.

2. ResNet-101 as Network Backbone

논문 저자들은 ResNet-101을 backbone으로 설정했을 때,
 다양한 inference 전략을 성능과 연산량 두가지 측면에 대해 실험하였습니다.
 아래는 이 실험을 테이블로 표현한 것입니다.
 실험 주요결과는 사진 위에 필기해뒀습니다.
 참고로 아래의 OS는 output stride를 뜻합니다.

Encoder		Decoder	MS	Flip	mIOU	Multiply-Adds
train OS	eval OS					
16	16				77.21%	81.02B
16	8				78.51%	276.18B
16	8		✓		79.45%	2435.37B
16	8		✓	✓	79.77%	4870.59B
16	16	✓			78.85%	101.28B
16	16	✓	✓		80.09%	898.69B
16	16	✓	✓	✓	80.22%	1797.23B
16	8	✓			79.35%	297.92B
16	8	✓	✓		80.43%	2623.61B
16	8	✓	✓	✓	80.57%	5247.07B
32	32				75.43%	52.43B
32	32	✓			77.37%	74.20B
32	16	✓			77.80%	101.28B
32	8	✓			77.92%	297.92B

TTA → MS는 어느정도 성능기여
Flip은 증가한 연산에 비해
성능향상은 약간 미미)

Decoder의 유무 (1)
: 있을시 더 좋은 성능

Coarser feature map (1)
: feature map이 더 정교할수록
좋은 성능

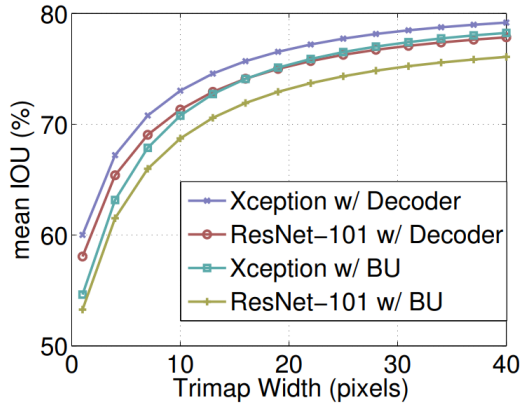
Coarser feature map (2)
: 성능향상은 있었지만
decoder 추가와 비교하면
증가한 연산에 비해 큰 성능
효과가 아님

Decoder의 유무 (2)
: 있을시 더 좋은 성능

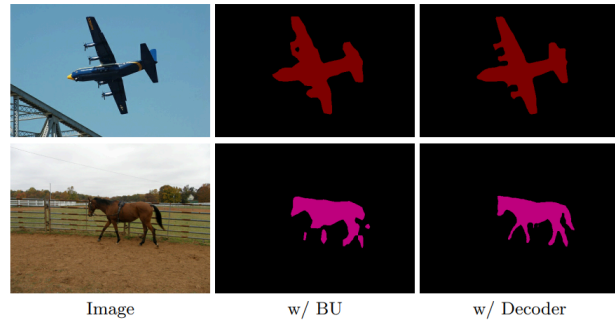
Table 3. Inference strategy on the PASCAL VOC 2012 *val* set using *ResNet-101*.
train OS: The *output stride* used during training. **eval OS:** The *output stride* used during evaluation. **Decoder:** Employing the proposed decoder structure. **MS:** Multi-scale inputs during evaluation. **Flip:** Adding left-right flipped inputs.

3. Improvement along Object Boundaries

앞에서는 DeepLabv3+는 segmentation을 할 때,
 더 정교한 경계선 정보를 얻기 위해 encoder-decoder 구조를 사용한다고 했습니다.
 논문에서는 단순히 bilinear upsampling을 했을 때와
 decoder를 사용했을 때의 성능을 비교해
 과연 실제로 encoder-decoder구조가 경계선 정보를 잘 표현하는지 실험했습니다.
 아래는 bilinear upsampling과 decoder를 사용했을 때의 성능을
 비교한 그래프와 사진입니다.
 (여기서 Trimap Width에 대해서는 소개하지 않겠습니다.)



(a) mIOU vs. Trimap width



(b) Decoder effect

Fig. 5. (a) mIOU as a function of trimap band width around the object boundaries when employing *train output stride* = *eval output stride* = 16. **BU**: Bilinear upsampling. (b) Qualitative effect of employing the proposed decoder module compared with the naive bilinear upsampling (denoted as **BU**). In the examples, we adopt Xception as feature extractor and *train output stride* = *eval output stride* = 16.

출처: <https://arxiv.org/pdf/1802.02611.pdf>

(a)에서 특정 Trimap Width를 잡고 mIOU값을 비교한다면

bilinear upsampling했을 때 보다 decoder를 사용했을 때가 더 성능이 좋을 수 있고,

(b)에서는 bilinear upsampling했을 때 보다 decoder를 사용했을 때가 더 경계선 정보를 잘 표현함을 알 수 있습니다.

4. Experimental Results on Cityscapes

아래 사진은 Cityscapes dataset에 대해

여러 모델의 testset mIOU값을 비교한 결과입니다.

(참고로 train dataset에는 coarse annotation도 포함해서 훈련시켰다고 합니다.)

Method	Coarse	mIOU
ResNet-38 [83]	✓	80.6
PSPNet [24]	✓	81.2
Mapillary [86]	✓	82.0
DeepLabv3	✓	81.3
DeepLabv3+	✓	82.1

(b) *test* set results

출처: <https://arxiv.org/pdf/1802.02611.pdf>

위에서 알 수 있듯이 DeepLabv3+는

논문이 나온 시점에서 Cityscapes dataset에 대해 sota를 차지했습니다.

[Conclusion]

드디어 Conclusion입니다!

지금까지 DeepLabv3+에 대해 살펴봤는데요.

수미상관같지만 ㅎㅎ DeepLabv3+는 아래와 같이 요약됩니다.

- ASPP(Atrous Spatial Pyramid Pooling)
- encoder-decoder structure
- depthwise separable convolution